

Programação Para Dispositivos Móveis II

UIKIT vs SWIFTUI

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

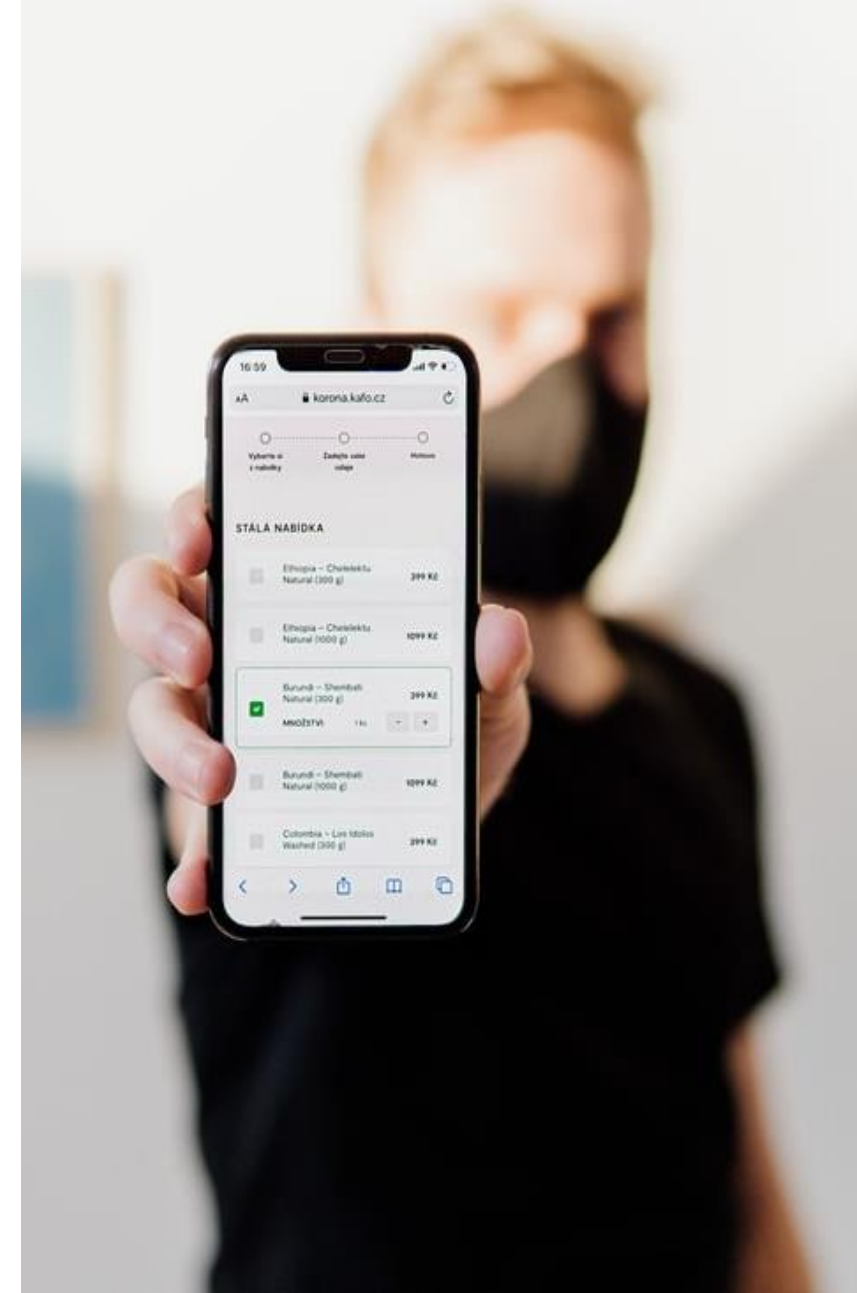
Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jrmr@estg.ipp.pt, Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt

UIKit vs SwiftUI

- Criar UI com Interface Builder e *Storyboards* (*UIKit*)
- Programar a UI com UIKit (imperativa)
- Programar a UI com SwiftUI (declarativa)



Roadmap

UIKit

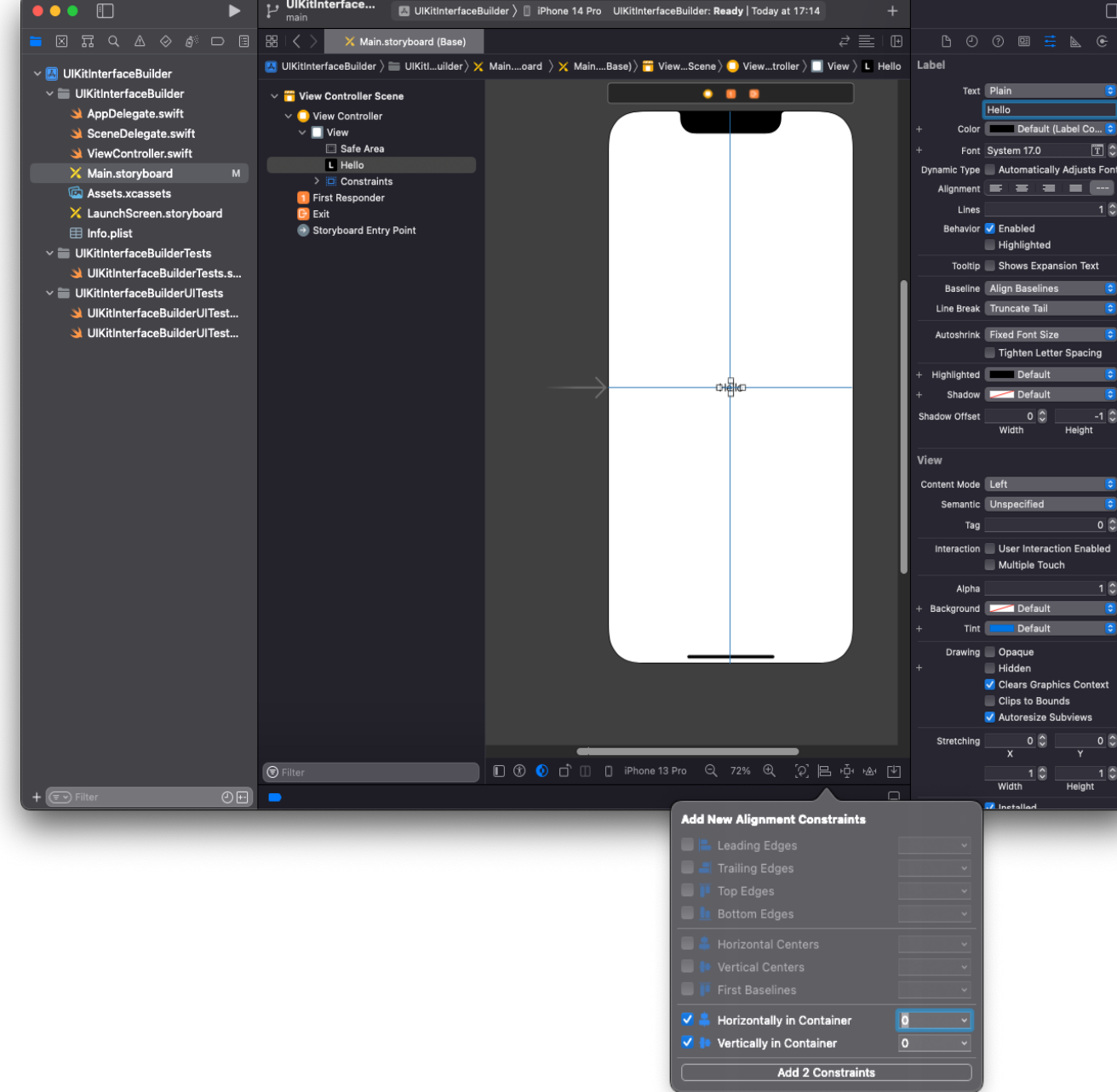


SwiftUI

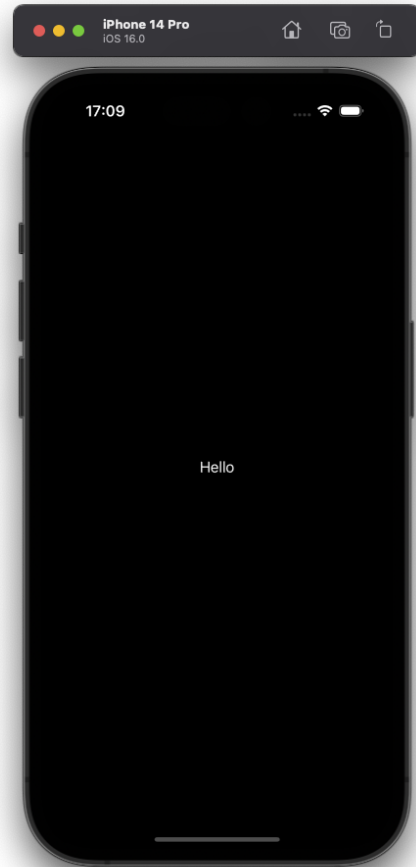


Interface Builder e Storyboards

- Adicionar a label através da componente library (+)
- Adicionar as restrições de alinhamento
- Alterar o texto no *Attributes Inspector*



UIKit programmatic UI



```
1 import UIKit
2
3 class ViewController: UIViewController {
4
5     override func viewDidLoad() {
6         super.viewDidLoad()
7         // Do any additional setup after loading the view.
8
9         // Create label
10        let label = UILabel()
11        label.text = "Hello"
12        label.textColor = UIColor.label
13
14        // don't create default constraints (we will handle that)
15        label.translatesAutoresizingMaskIntoConstraints = false
16        self.view.addSubview(label)
17
18        // Create constraints
19        let centerX = NSLayoutConstraint(item: label, attribute:
20            .centerX, relatedBy: .equal, toItem: self.view,
21            attribute: .centerX, multiplier: 1, constant: 0)
22        let centerY = NSLayoutConstraint(item: label, attribute:
23            .centerY, relatedBy: .equal, toItem: self.view,
24            attribute: .centerY, multiplier: 1, constant: 0)
25
26        self.view.addConstraints([centerX, centerY])
27    }
28 }
```

Roadmap

UIKit

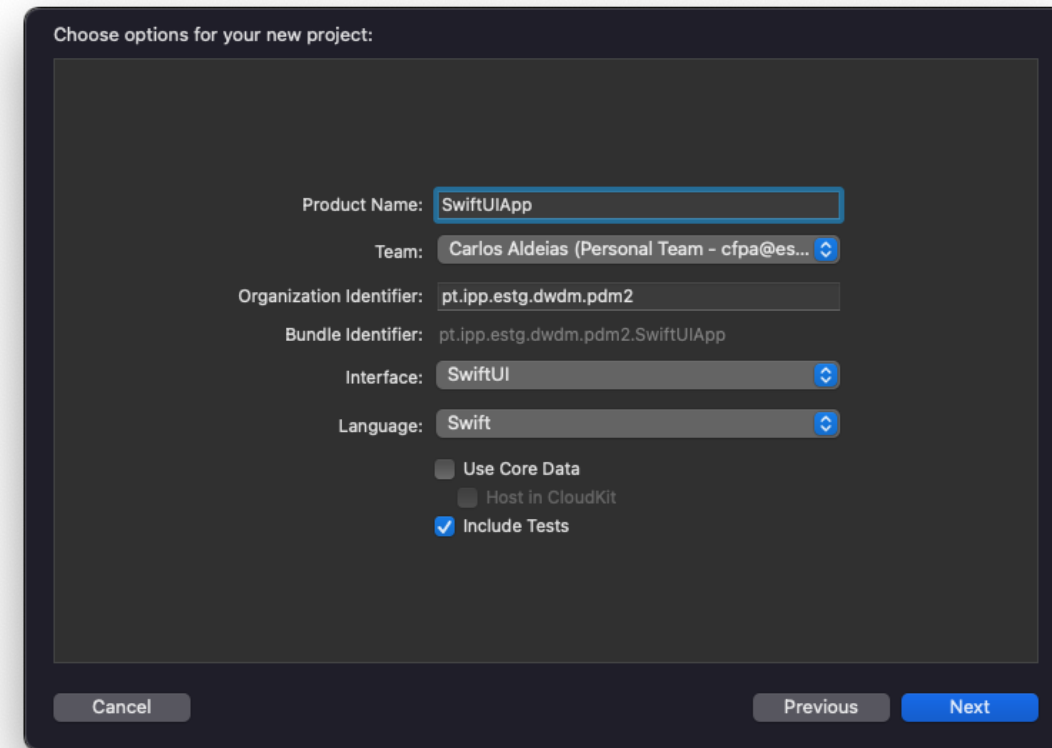


SwiftUI

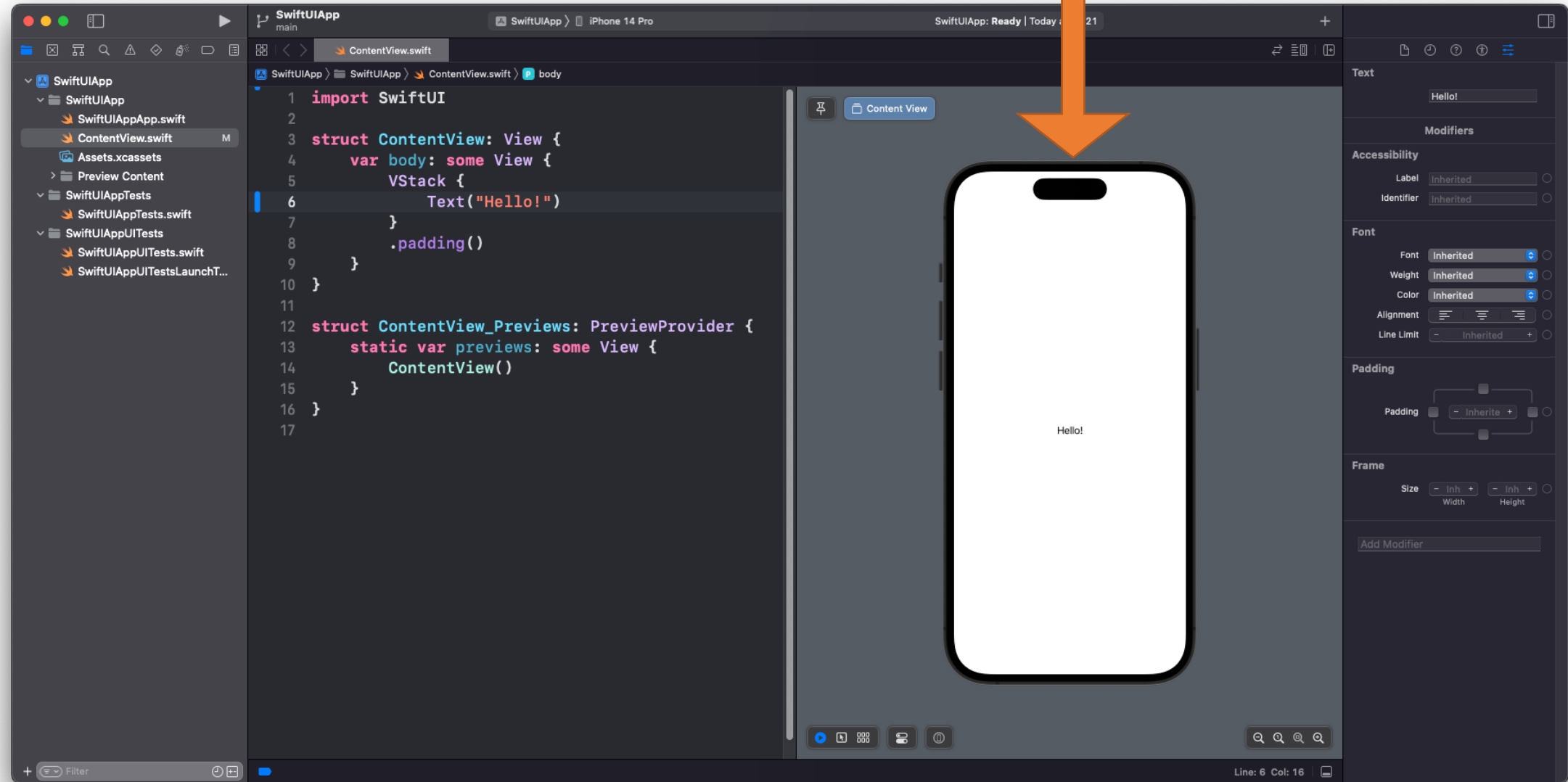


SwiftUI app

- Ao criar o projeto, em Interface, selecionar SwiftUI
- Não existe o *ViewController*
- Não existem storyboards
- Prevalência do **MVVM**
- Acesso ao ***Preview*** (pré-visualização)
- Permite alterações no objetos da UI através do *Attributes Inspector*, que insere código automaticamente no *Content View*



SwiftUI app



MVVM Architectural Pattern

- **VIEW**

- Representa aquilo que o utilizador vê no ecrã
- Mostra informação
- Permite a interação com o utilizador

- **VIEWMODEL**

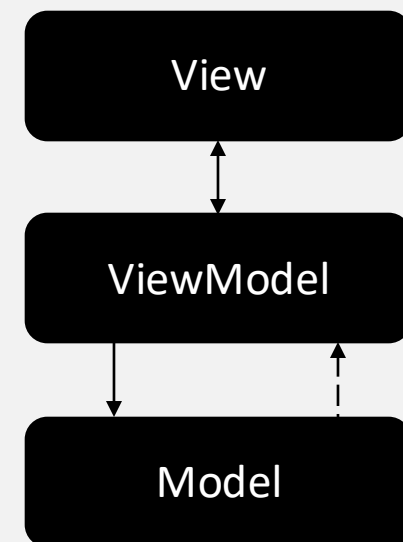
- Contém o estado atual da vista
- Conecta a outras partes da arquitetura da app (ex, armazenamento de dados ou acesso à rede)
- Suporte para a interação com o utilizador

- **MODEL**

- Dados da aplicação e da lógica do negócio

A Apple seguiu desde cedo o padrão **MVC** para aplicações em **macOS** e **iOS**.
Esta estratégia mudou com a introdução do **SwiftUI**.

MVVM Architectural Pattern



Uma das principais diferenças para o **MVC** é que o **MVVM conecta** as camadas da *View* e *ViewModel* através de **Binders**, mantendo a sincronização de dados

@State e @Binding

- Utilizar **@State** (na parent view), para indicar qual é a propriedade que armazena os dados a partilhar
 - Atributos privados (uso de **private**)
- Utilizar **@Binding** para criar uma ligação bidirecional
 - entre uma propriedade que armazena dados...
 - ... e a vista que mostra e modifica esses dados

```
struct PlayerView: View {
    var episode: Episode
    @State private var isPlaying: Bool = false

    var body: some View {
        VStack {
            Text(episode.title)
                .foregroundStyle(isPlaying ? .primary : .secondary)
            PlayButton(isPlaying: $isPlaying) // Pass a binding.
        }
    }
}
```



```
struct PlayButton: View {
    @Binding var isPlaying: Bool

    var body: some View {
        Button(isPlaying ? "Pause" : "Play") {
            isPlaying.toggle()
        }
    }
}
```

@StateObject e @EnvironmentObject

- Utilizar **@StateObject** para criar e gerir uma instância de um objeto numa vista
 - Pode também ser usado ao nível mais alto da App
- Utilizar **@EnvironmentObject** para partilhar um objeto em toda a aplicação
 - Adicionar também o *modifier* **.environmentObject** no *preview provider*

```
struct LibraryView: View {  
    @StateObject private var book = Book()  
  
    var body: some View {  
        BookView(book: book)  
    }  
}
```

```
@main  
struct BookReader: App {  
    @StateObject private var library = Library()  
  
    // ...  
}
```

```
@main  
struct BookReader: App {  
    @StateObject private var library = Library()  
  
    var body: some Scene {  
        WindowGroup {  
            LibraryView()  
                .environmentObject(library)  
        }  
    }  
}
```

```
struct LibraryView: View {  
    @EnvironmentObject var library: Library  
  
    // ...  
}
```

SwiftUI

Gestão do Model na aplicação

- O model deve adotar o protocolo **ObservableObject**
- Utilizar **@Published** para partilhar uma propriedade que emitirá/enviará o novo valor sempre que for alterado
 - Só devem ser publicados os atributos relevantes para a UI e que são atualizados por ambos (por exemplo, o *identifier* nunca muda, logo não deve ser **@Published**)
- Utilizar **@ObservedObject** para indicar que um objeto (**ObservableObject**) deve ser monitorado.
 - Sempre que os dados partilhados desse objeto alterarem, o **SwiftUI** atualiza as vistas associadas

```
class Book: ObservableObject {  
}
```

```
class Book: ObservableObject {  
    @Published var title = "Great Expectations"  
  
    let identifier = UUID() // A unique identifier that never changes.  
}
```

```
struct BookView: View {  
    @ObservedObject var book: Book  
  
    var body: some View {  
        BookEditView(book: book)  
    }  
}  
  
struct BookEditView: View {  
    @ObservedObject var book: Book  
  
    // ...  
}
```

SwiftUI

Ligação bidirecional com *bindings*

- Permite ao utilizador alterar os dados na UI
- Deve-se fazer o **binding** à propriedade (no exemplo, ao **.title**)
- Pode aplicar binding a atributos de:
 - **@StateObject**
 - **@ObservedObject**
 - **@EnvironmentObject**

Acrescentar o
prefixo \$

```
struct BookEditView: View {  
    @ObservedObject var book: Book  
  
    var body: some View {  
        TextField("Title", text: $book.title)  
    }  
}
```

Next Milestone

- Persistência de Dados
 - User Defaults
 - Ficheiros
 - Core Data Framework



Leitura recomendada

- **UIKit Framework** <https://developer.apple.com/documentation/uikit>
- **SwiftUI Framework** <https://developer.apple.com/documentation/swiftui/>
- **Binding** <https://developer.apple.com/documentation/swiftui/binding>
- **Managing model data in your app** <https://developer.apple.com/documentation/swiftui/managing-model-data-in-your-app>

Programação Para Dispositivos Móveis II

UIKIT vs SWIFTUI

2025/26 CTeSP – Desenvolvimento para a Web e Dispositivos Móveis

Carlos Aldeias, cfpa@estg.ipp.pt

José Teixeira, jmt@estg.ipp.pt

Adaptação do conteúdo dos slides de João Ramos jrmr@estg.ipp.pt , Fábio Silva fas@estg.ipp.pt e Ricardo Barbosa rmb@estg.ipp.pt